МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №1

з дисципліни **"Функціональне програмування"**

на тему **"ОСНОВИ ФУНКЦІОНАЛЬНОГО ПРОГРАМУВАННЯ У PYTHON"**

Варіант 14. Вартість навчальних курсів

Виконала:
Студентка групи ФЕП-23
Поведа Анастасія

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: засвоїти поняття чистої функції та референтної прозорості, навчитися знаходити й ізолювати побічні ефекти, переписати імперативний розрахунок вартості навчальних курсів у функціональному стилі та зробити правила знижок і податку взаємозамінними за допомогою `typing.Callable`.

Теоретичні відомості:

Функцію називають чистою, якщо її результат залежить лише від аргументів, вона не змінює ані глобального стану, ані переданих їй даних і не виконує введення-виведення. Код із чистих функцій є референтно прозорим: будь-який виклик можна замінити його значенням, і програма поводитиметься так само. Завдяки цьому такі функції легко тестувати ізольовано, кешувати та переставляти місцями.

До побічних ефектів належать друк і робота з файлами чи мережею, мутація вхідних списків і словників, читання або зміна глобальних змінних, залежність від поточного часу або випадкових чисел. У функціональному підході програму ділять на чисте ядро, яке лише перетворює дані, і тонку оболонку, яка читає вхід, викликає ядро та виводить результат. Зовнішні залежності (годинник, джерело випадковості) передають у ядро як функції, а змінні правила обчислень (студентська знижка, пакетна знижка, податок, фільтр за ціною) — як значення типу `Callable`. Тоді правила можна компонувати: кілька дрібних функцій знижки об’єднуються в одну, а будь-яку з них можна замінити, не змінюючи саме ядро.

Хід роботи:

1. За варіантом 14 (вартість навчальних курсів: курс, ціна, статус слухача, кількість курсів) написала стартовий імперативний фрагмент у `starter_imperative.py`, що для підтверджених реєстрацій рахує кінцеву вартість зі студентською і пакетною знижками та податком. У ньому зібрані ті самі проблеми, що й у прикладі з методички: глобальні константи `STATUS_DISCOUNTS`, `BUNDLE_DISCOUNT`, `MAX_DISCOUNT`, `TAX_RATE`, виклик `print`

посеред обчислень, мутація вхідних словників (`e["cost"] = …`) і змішування розрахунку з I/O:

```python
def process_enrollments(enrollments: list[Any], min_price: float) -> dict[str, Any]:
    valid: list[dict[str, Any]] = []
    total_cost = 0.0

    for e in enrollments:
        if not e["confirmed"]:
            continue
        print("Processing enrollment:", e["id"])

        price = e["price"]
        if price < min_price:
            continue

        discount = STATUS_DISCOUNTS[e["status"]]
        if e["courses"] >= BUNDLE_MIN_COURSES:
            discount += BUNDLE_DISCOUNT
        if discount > MAX_DISCOUNT:
            discount = MAX_DISCOUNT
        cost = price * (1 - discount)
        cost = cost * (1 + TAX_RATE)

        e["cost"] = cost
        valid.append(e)
        total_cost += cost

    return {"count": len(valid), "total_cost": total_cost, "enrollments": valid}
```

2. У файлі `core.py` описала типи даних через `TypedDict` (`Enrollment`, `Report`), псевдоніми правил на основі `Callable` (зокрема `DiscountRule` — функція від реєстрації до частки знижки) і дві базові чисті функції: `final_cost` обчислює вартість як ціну з урахуванням знижки та податку, а `with_cost` повертає новий словник, не чіпаючи вхідний:

```python
class Enrollment(TypedDict, total=False):
    id: int
    course: str
    price: float
    status: str
    courses: int
    confirmed: bool
    cost: float


class Report(TypedDict):
    count: int
    total_cost: float
    enrollments: list[Enrollment]


FilterFn = Callable[[float], bool]
DiscountRule = Callable[[Enrollment], float]
AmountFn = Callable[[float], float]
EnrollmentProcessor = Callable[[Iterable[Enrollment]], Report]
NowFn = Callable[[], float]

A = TypeVar("A")
B = TypeVar("B")
C = TypeVar("C")


def final_cost(price: float, discount: float, tax_rate: float) -> float:
    """Кінцева вартість: ціна × (1 − знижка) × (1 + податок)."""
    return price * (1 - discount) * (1 + tax_rate)


def with_cost(enrollment: Enrollment, cost: float) -> Enrollment:
    """Повертає НОВУ реєстрацію з полем cost (вхідна не змінюється)."""
    return {**enrollment, "cost": cost}
```

3. Реалізувала фабрику `make_enrollment_processor`, що приймає три правила (`accept` — фільтр за ціною, `discount` — сумарна знижка для реєстрації, `apply_tax` — податок) і повертає налаштований обробник; він відбирає підтверджені реєстрації через list comprehension і створює нові записи з полем `cost`. Правила знижок зроблені компонувальними: `status_discount` і `bundle_discount` — окремі функції, а `combine_discounts` складає будь-яку їх кількість в одне правило з обмеженням `cap`. Функція `process_enrollments_pure` збирає обробник із цих фабрик, отримуючи всі параметри (таблицю знижок за статусом, поріг пакета, максимальну знижку, ставку податку) явно:

```python
def make_enrollment_processor(
    *,
    accept: FilterFn,
    discount: DiscountRule,
    apply_tax: AmountFn,
) -> EnrollmentProcessor:
    """Фабрика обробника реєстрацій: усі правила передаються як Callable."""

    def process(enrollments: Iterable[Enrollment]) -> Report:
        """Відбирає підтверджені реєстрації, що пройшли accept, і рахує cost."""
        qualified = [
            with_cost(e, apply_tax(e["price"] * (1 - discount(e))))
            for e in enrollments
            if e["confirmed"] and accept(e["price"])
        ]
        return {
            "count": len(qualified),
            "total_cost": sum(e["cost"] for e in qualified),
            "enrollments": qualified,
        }

    return process

```

```python
def min_price_filter(min_price: float) -> FilterFn:
    """Правило фільтра: приймати курси з ціною не нижче порога."""
    return lambda price: price >= min_price


def status_discount(rates: Mapping[str, float], default: float = 0.0) -> DiscountRule:
    """Студентська знижка: частка з таблиці за статусом, інакше default."""
    return lambda e: rates.get(e["status"], default)


def bundle_discount(min_courses: int, rate: float) -> DiscountRule:
    """Пакетна знижка: rate, якщо куплено не менше min_courses курсів."""
    return lambda e: rate if e["courses"] >= min_courses else 0.0


def combine_discounts(*rules: DiscountRule, cap: float) -> DiscountRule:
    """Компонує правила знижок: сума знижок, але не більше cap."""
    return lambda e: min(sum(rule(e) for rule in rules), cap)


def tax(rate: float) -> AmountFn:
    """Правило податку: додає rate до суми."""
    return lambda amount: amount * (1 + rate)


def process_enrollments_pure(
    enrollments: Iterable[Enrollment],
    *,
    min_price: float,
    status_rates: Mapping[str, float],
    bundle_min: int,
    bundle_rate: float,
    max_discount: float,
    tax_rate: float,
) -> Report:
    """Чистий розрахунок вартості курсів з явними параметрами (без глобалів)."""
    process = make_enrollment_processor(
        accept=min_price_filter(min_price),
        discount=combine_discounts(
            status_discount(status_rates),
            bundle_discount(bundle_min, bundle_rate),
            cap=max_discount,
        ),
        apply_tax=tax(tax_rate),
    )
    return process(enrollments)
```

4. Додала функції з додаткових вправ: узагальнену композицію `compose`, конвеєр `pipe`, фабрику множників `make_multiplier` і `stamp_total`, якій годинник передається ззовні як `Callable`, тому вона лишається тестованою без реального часу:

```python
def compose(f: Callable[[B], C], g: Callable[[A], B]) -> Callable[[A], C]:
    """Композиція справа наліво: compose(f, g)(x) == f(g(x))."""
    return lambda x: f(g(x))


def pipe(x: A, *funcs: Callable[[A], A]) -> A:
    """Конвеєр зліва направо: pipe(x, f, g) == g(f(x))."""
    for f in funcs:
        x = f(x)
    return x


def make_multiplier(k: int) -> Callable[[int], int]:
    """Фабрика множників: make_multiplier(3)(7) == 21."""
    return lambda x: x * k


def stamp_total(total: float, now: NowFn) -> tuple[float, float]:
    """Додає часову позначку; джерело часу ін'єктується ззовні."""
    return total, now()
```

5. Базове завдання з методички (обробка замовлень) також переписала у функціональному стилі в тому ж `core.py`: `order_subtotal`, `with_total`, фабрика

`make_processor` з політиками `accept`, `apply_discount`, `apply_tax` та `process_orders_pure` з явними параметрами `min_total`, `discount`, `tax_rate`:

```python
class Item(TypedDict):
    price: float
    qty: int


class Order(TypedDict, total=False):
    id: int
    items: list[Item]
    paid: bool
    total: float


class OrderReport(TypedDict):
    count: int
    revenue: float
    orders: list[Order]


Processor = Callable[[Iterable[Order]], OrderReport]


def order_subtotal(order: Order) -> float:
    """Сума позицій замовлення (базове завдання)."""
    return sum(it["price"] * it["qty"] for it in order["items"])


def with_total(order: Order, total: float) -> Order:
    """Повертає НОВЕ замовлення з полем total."""
    return {**order, "total": total}


def make_processor(
    *,
    accept: FilterFn,
    apply_discount: AmountFn,
    apply_tax: AmountFn,
) -> Processor:
    """Фабрика обробника замовлень із політиками accept / apply_discount / apply_tax."""

    def process(orders: Iterable[Order]) -> OrderReport:
        """Відбирає оплачені замовлення, що пройшли accept, і рахує total."""
        qualified = [
            with_total(o, apply_tax(apply_discount(order_subtotal(o))))
            for o in orders
            if o["paid"] and accept(order_subtotal(o))
        ]
        return {
            "count": len(qualified),
            "revenue": sum(o["total"] for o in qualified),
            "orders": qualified,
        }

    return process


def process_orders_pure(
    orders: Iterable[Order],
    *,
    min_total: float,
    discount: float,
    tax_rate: float,
) -> OrderReport:
    """Чиста версія стартового process_orders: усі параметри явні."""
    process = make_processor(
        accept=lambda subtotal: subtotal >= min_total,
        apply_discount=lambda amount: amount * (1 - discount),
        apply_tax=lambda amount: amount * (1 + tax_rate),
    )
    return process(orders)
```

6. Усі побічні ефекти винесла в оболонку `app.py`: читання реєстрацій із JSON, функцію `render_report`, що лише перетворює чистий результат на текст,

функцію `run` для базового завдання, друк і передачу `time.time` у ядро:

```python
def load_enrollments(path: Path) -> list[Enrollment]:
    """IO-адаптер: читає реєстрації з JSON-файлу."""
    return list(json.loads(path.read_text(encoding="utf-8")))


def render_report(result: Report) -> str:
    """Перетворює чистий результат на текст звіту (без друку)."""
    lines = [
        f"Enrollment: {e['id']} {e['course']} cost: {e['cost']:.2f}"
        for e in result["enrollments"]
    ]
    return "\n".join(
        [*lines, f"Count: {result['count']}", f"Total: {result['total_cost']:.2f}"]
    )


def run(orders: list[Order]) -> None:
    """Оболонка базового завдання: друкує чистий результат process_orders_pure."""
    result = process_orders_pure(orders, min_total=100, discount=0.1, tax_rate=0.2)
    for o in result["orders"]:
        print("Processed:", o["id"], "total:", o["total"])
    print("Revenue:", result["revenue"])


def main(argv: list[str]) -> None:
    """Точка входу: читання → чисте ядро → друк; час передається в ядро."""
    enrollments = load_enrollments(Path(argv[1])) if len(argv) > 1 else SAMPLE
    result = process_enrollments_pure(
        enrollments,
        min_price=1000,
        status_rates=STATUS_DISCOUNTS,
        bundle_min=3,
        bundle_rate=0.1,
        max_discount=0.25,
        tax_rate=0.2,
    )
    print(render_report(result))
    total, stamp = stamp_total(result["total_cost"], time.time)
    print(f"Stamped: {total:.2f} at {time.strftime('%H:%M:%S', time.localtime(stamp))}")


if __name__ == "__main__":
    main(sys.argv)
```

7. Написала pytest-тести у `tests/test_core.py`: референтна прозорість, відсутність мутації вхідних даних, еквівалентність імперативній версії,

відсіювання непідтверджених і дешевих реєстрацій, перевірка `final_cost` та `with_cost`:

```python
def run(enrollments: list[Enrollment]) -> Report:
    return process_enrollments_pure(
        enrollments,
        min_price=1000.0,
        status_rates=RATES,
        bundle_min=3,
        bundle_rate=0.1,
        max_discount=0.25,
        tax_rate=0.2,
    )


def test_referential_transparency(sample_enrollments: list[Enrollment]) -> None:
    r1 = run(sample_enrollments)
    r2 = run(sample_enrollments)
    assert r1 == r2


def test_no_mutation(sample_enrollments: list[Enrollment]) -> None:
    original = deepcopy(sample_enrollments)
    process_enrollments_pure(
        sample_enrollments,
        min_price=0,
        status_rates=RATES,
        bundle_min=1,
        bundle_rate=0.5,
        max_discount=1.0,
        tax_rate=0.0,
    )
    assert sample_enrollments == original


def test_matches_imperative(sample_enrollments: list[Enrollment]) -> None:
    expected = process_enrollments(deepcopy(sample_enrollments), 1000)
    assert run(sample_enrollments) == expected


def test_filters_unconfirmed_and_cheap(sample_enrollments: list[Enrollment]) -> None:
    result = run(sample_enrollments)
    assert [e["id"] for e in result["enrollments"]] == [1, 4, 5]
    assert result["count"] == 3
    assert result["total_cost"] == pytest.approx(
        (4000 * 0.85 + 9000 * 0.75 + 2000 * 0.85) * 1.2
    )

```

8. Додала тести на Callable-правила (`make_enrollment_processor` з лямбдами), окремі правила знижок, композицію знижок з обмеженням `cap`, `compose` і

`make_multiplier`, інжекцію годинника та текст звіту, а в `tests/test_orders.py` — мінімальні тести з методички для базового завдання:

```python
def test_callable_policies(sample_enrollments: list[Enrollment]) -> None:
    process = make_enrollment_processor(
        accept=lambda price: price >= 3000,
        discount=lambda e: 0.5 if e["status"] == "student" else 0.0,
        apply_tax=lambda amount: amount,
    )
    result = process(sample_enrollments)
    assert [e["id"] for e in result["enrollments"]] == [1, 4]
    assert result["total_cost"] == pytest.approx(4000 * 0.5 + 9000 * 0.5)

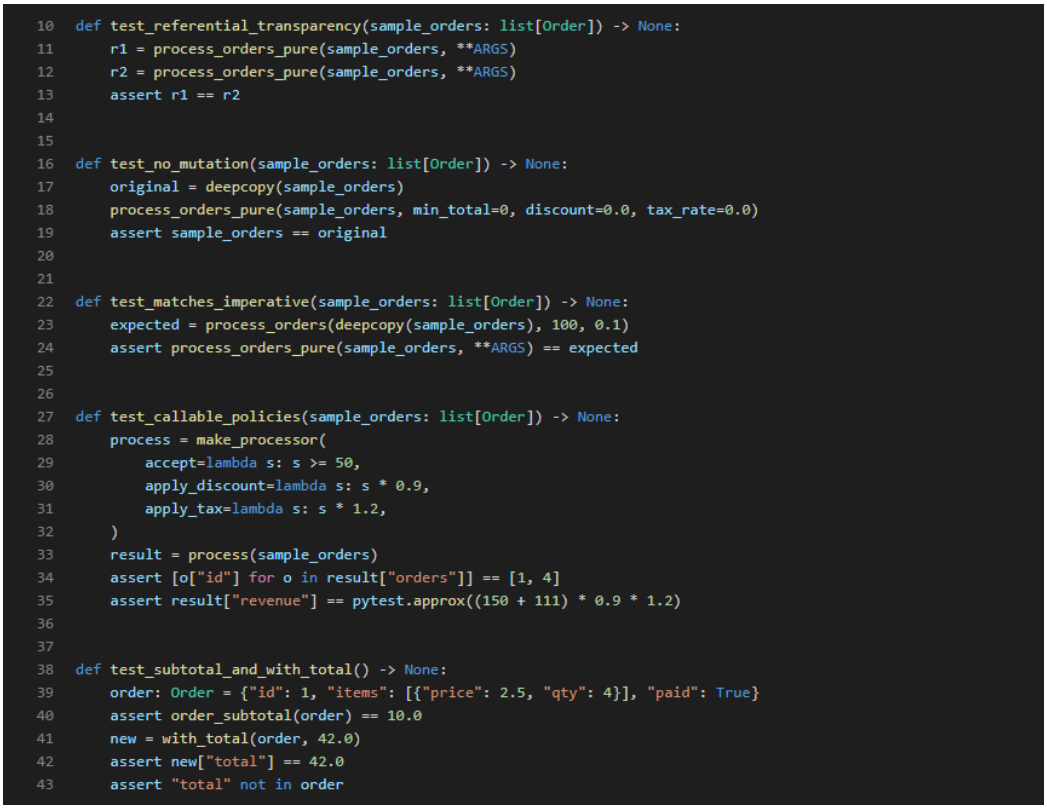

def test_discount_rules() -> None:
    student: Enrollment = {"status": "student", "courses": 1}
    bulk: Enrollment = {"status": "unknown", "courses": 5}
    assert min_price_filter(500)(499.9) is False
    assert min_price_filter(500)(500) is True
    assert status_discount(RATES)(student) == 0.15
    assert status_discount(RATES, default=0.01)(bulk) == 0.01
    assert bundle_discount(3, 0.1)(student) == 0.0
    assert bundle_discount(3, 0.1)(bulk) == 0.1
    assert tax(0.2)(100.0) == pytest.approx(120.0)


def test_combine_discounts_is_capped() -> None:
    rule = combine_discounts(status_discount(RATES), bundle_discount(3, 0.1), cap=0.2)
    assert rule({"status": "student", "courses": 1}) == pytest.approx(0.15)
    assert rule({"status": "student", "courses": 3}) == pytest.approx(0.2)
    assert rule({"status": "regular", "courses": 3}) == pytest.approx(0.1)


def test_compose_and_multiplier() -> None:
    assert make_multiplier(3)(7) == 21
    assert compose(make_multiplier(2), make_multiplier(5))(1) == 10
    assert pipe(1, make_multiplier(2), make_multiplier(5)) == 10
    priced = compose(tax(0.2), tax(-0.1))
    assert priced(100.0) == pytest.approx(108.0)


def test_stamp_total_uses_injected_clock() -> None:
    assert stamp_total(10.0, lambda: 123.0) == (10.0, 123.0)


def test_render_report(sample_enrollments: list[Enrollment]) -> None:
    text = render_report(run(sample_enrollments))
    assert text.splitlines() == [
        "Enrollment: 1 Python cost: 4080.00",
        "Enrollment: 4 Data Science cost: 8100.00",
        "Enrollment: 5 Go cost: 2040.00",
        "Count: 3",
        "Total: 14220.00",
    ]
```



```python
def test_referential_transparency(sample_orders: list[Order]) -> None:
    r1 = process_orders_pure(sample_orders, **ARGS)
    r2 = process_orders_pure(sample_orders, **ARGS)
    assert r1 == r2


def test_no_mutation(sample_orders: list[Order]) -> None:
    original = deepcopy(sample_orders)
    process_orders_pure(sample_orders, min_total=0, discount=0.0, tax_rate=0.0)
    assert sample_orders == original


def test_matches_imperative(sample_orders: list[Order]) -> None:
    expected = process_orders(deepcopy(sample_orders), 100, 0.1)
    assert process_orders_pure(sample_orders, **ARGS) == expected


def test_callable_policies(sample_orders: list[Order]) -> None:
    process = make_processor(
        accept=lambda s: s >= 50,
        apply_discount=lambda s: s * 0.9,
        apply_tax=lambda s: s * 1.2,
    )
    result = process(sample_orders)
    assert [o["id"] for o in result["orders"]] == [1, 4]
    assert result["revenue"] == pytest.approx((150 + 111) * 0.9 * 1.2)


def test_subtotal_and_with_total() -> None:
    order: Order = {"id": 1, "items": [{"price": 2.5, "qty": 4}], "paid": True}
    assert order_subtotal(order) == 10.0
    new = with_total(order, 42.0)
    assert new["total"] == 42.0
    assert "total" not in order
```

9. У `tests/test_properties.py` додала property-based тести на Hypothesis: для випадкових списків реєстрацій перевіряються референтна прозорість, відсутність мутацій та інваріанти підсумку (кількість дорівнює числу

підтверджених реєстрацій, загальна сума — сумі вартостей, а вартість лежить у межах від ціни з максимальною знижкою до ціни без знижки з податком):

```python
def run(items: list[Enrollment]) -> Report:
    return process_enrollments_pure(
        items,
        min_price=0.0,
        status_rates=RATES,
        bundle_min=3,
        bundle_rate=0.1,
        max_discount=0.25,
        tax_rate=0.2,
    )


enrollments = st.lists(
    st.fixed_dictionaries(
        {
            "id": st.integers(min_value=1),
            "course": st.text(min_size=1, max_size=10),
            "price": st.floats(min_value=0, max_value=1e6, allow_nan=False),
            "status": st.sampled_from(["student", "regular", "alumni"]),
            "courses": st.integers(min_value=1, max_value=20),
            "confirmed": st.booleans(),
        }
    ),
    max_size=20,
)


@given(enrollments)
def test_property_referential_transparency(items: list[Enrollment]) -> None:
    assert run(items) == run(items)


@given(enrollments)
def test_property_no_mutation(items: list[Enrollment]) -> None:
    original = deepcopy(items)
    run(items)
    assert items == original


@given(enrollments)
def test_property_total_is_sum_and_only_confirmed(items: list[Enrollment]) -> None:
    result = run(items)
    assert result["count"] == sum(1 for e in items if e["confirmed"])
    assert result["total_cost"] == sum(e["cost"] for e in result["enrollments"])
    assert all(
        e["price"] * 0.75 * 1.2 - 1e-6 <= e["cost"] <= e["price"] * 1.2 + 1e-6
        for e in result["enrollments"]
    )
```

10. Запустила оболонку на вбудованому прикладі: з чотирьох реєстрацій відібрано дві підтверджені з ціною не менше 1000, до них застосовано студентську знижку 15 %, пакетну 10 % за три і більше курсів (разом не більше 25 %) і податок 20 %:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab01> python app.py
Enrollment: 1 Python cost: 4080.00
Enrollment: 4 Data Science cost: 8100.00
Count: 2
Total: 12180.00
Stamped: 12180.00 at 17:20:15
```

11. Перевірила проєкт автоматичними інструментами: усі 20 тестів pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають

зауважень до стилю:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab01> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs_functional_for_nastia
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 20 items

tests\test_core.py::test_referential_transparency PASSED                [  5%]
tests\test_core.py::test_no_mutation PASSED                             [ 10%]
tests\test_core.py::test_matches_imperative PASSED                      [ 15%]
tests\test_core.py::test_filters_unconfirmed_and_cheap PASSED           [ 20%]
tests\test_core.py::test_final_cost PASSED                              [ 25%]
tests\test_core.py::test_with_cost_returns_new_dict PASSED              [ 30%]
tests\test_core.py::test_callable_policies PASSED                       [ 35%]
tests\test_core.py::test_discount_rules PASSED                          [ 40%]
tests\test_core.py::test_combine_discounts_is_capped PASSED             [ 45%]
tests\test_core.py::test_compose_and_multiplier PASSED                  [ 50%]
tests\test_core.py::test_stamp_total_uses_injected_clock PASSED         [ 55%]
tests\test_core.py::test_render_report PASSED                           [ 60%]
tests\test_orders.py::test_referential_transparency PASSED              [ 65%]
tests\test_orders.py::test_no_mutation PASSED                           [ 70%]
tests\test_orders.py::test_matches_imperative PASSED                    [ 75%]
tests\test_orders.py::test_callable_policies PASSED                     [ 80%]
tests\test_orders.py::test_subtotal_and_with_total PASSED               [ 85%]
tests\test_properties.py::test_property_referential_transparency PASSED [ 90%]
tests\test_properties.py::test_property_no_mutation PASSED              [ 95%]
tests\test_properties.py::test_property_total_is_sum_and_only_confirmed PASSED [100%]

============================= 20 passed in 0.95s ==============================

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab01> python -m mypy .
Success: no issues found in 7 source files

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab01> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab01> python -m black --check .
All done! ✨ 🍰 ✨
7 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я розібралася з поняттями чистої функції, референтної прозорості та побічних ефектів і переписала імперативний розрахунок вартості навчальних курсів (варіант 14) у функціональному стилі: прибрала глобальний стан, мутації вхідних реєстрацій і `print` із ядра, винесла I/O в окрему оболонку, а правила студентської та пакетної знижок, податку й фільтра передала через `typing.Callable` у фабрику `make_enrollment_processor`, причому знижки компонуються функцією `combine_discounts`. Юніт- і property-тести на референтну прозорість і відсутність мутацій, а також перевірки mypy, ruff і black підтвердили коректність і чистоту реалізації.